Multi-Agent Pathfinding in Warehouse Environments

Using Conflict-Based Search with A for Collision-Free Multi-Robot Navigation*

1. PROBLEM STATEMENT

Modern automated warehouses deploy fleets of autonomous mobile robots (AMRs) to retrieve and transport goods. As fleet sizes grow, a fundamental challenge emerges: how can multiple robots navigate a shared floor plan simultaneously, reaching their individual destinations as efficiently as possible without colliding with each other?

This problem is known as Multi-Agent Pathfinding (MAPF). The naive solution — plan each robot's path independently with single-agent A* — fails because it ignores inter-robot conflicts: two robots may be assigned paths that require them to occupy the same cell at the same time, or to swap positions in a single time step.

This project implements and evaluates Conflict-Based Search (CBS), a provably optimal MAPF algorithm that uses A* as its core subroutine, on a warehouse-inspired grid environment. The key engineering question is: how does CBS scale in solution quality and computation time compared to a simpler prioritized planning baseline as the number of agents increases?

2. CHOSEN TECHNIQUE

Conflict-Based Search (CBS) operates at two levels. The **high level** maintains a constraint tree (CT) where each node represents a set of constraints imposed on individual agents and the combined cost of their paths. The **low level** runs single-agent A* for each agent subject to its assigned constraints, using Manhattan distance as the admissible heuristic.

The CBS algorithm proceeds as follows:

- Compute an initial plan: run A* independently for each agent (no constraints).
- Detect the first conflict — any pair of agents occupying the same cell at the same time step, or swapping positions between consecutive steps.
- Branch: create two child CT-nodes, each adding one constraint that resolves the conflict (agent i must not be at cell c at time t, or agent j must not).
- Replan the constrained agent at the low level using A* with the new constraint set.
- Expand the CT node with the lowest total cost (sum of all individual path costs). Repeat until a conflict-free solution is found.

CBS is complete and optimal: it is guaranteed to find a solution if one exists, and it minimises the sum-of-costs objective. Its correctness follows directly from the optimality of A* at the low level and the exhaustive branching at the high level.

3. ENVIRONMENT AND DATASET

The environment is a synthetic warehouse grid world generated programmatically — no external dataset download is required. The grid consists of open floor cells and static obstacle cells (representing racks and walls), and is fully reproducible via a fixed random seed.

Parameter	Value / Description
Grid size	16 × 16 cells (scalable to 32 × 32)
Obstacle density	~20% of cells blocked (warehouse racks)
Number of agents	Swept from 2 to 20 in the scaling experiment
Start/goal pairs	Randomly sampled, guaranteed reachable
Cost metric	Sum of individual path lengths (sum-of-costs)
Baseline compared	Prioritized A* (agents planned in priority order)
Time limit per run	30 seconds (CBS) — reports timeout if exceeded

4. PROJECT PLAN AND MILESTONES

Week	Task	Deliverable
14	Project setup. Start to implement grid environment, single-agent A* with Manhattan heuristic, conflict detection, and CT node structure.	Proposal (this document)
15	Implement full CBS high-level search. Implement Prioritized A* baseline. Run correctness tests: verify optimal cost on small known instances.	Working A* + conflict detector Working CBS + baseline + unit tests
16	Run scaling experiments (2–20 agents). Generate plots: cost vs. agents, runtime vs. agents, CT nodes expanded vs. agents. Write final report.	Final report (IEEE format, 5–10 pp.) + complete code repository with README

5. EXPECTED OUTCOMES AND EVALUATION

The primary risk involves the computational scalability of CBS as the fleet size increases toward 20 robots. Given the 30-second time limit, there is a distinct possibility that the algorithm will exceed this threshold due to a high volume of conflicts in the 16 x 16 environment. Consequently, the project must address whether CBS becomes computationally prohibitive, identifying the "break points" where the pursuit of optimality outweighs the practical need for a timely solution in a real-time warehouse deployment.

The project should produce three concrete results:

- Correctness verification: CBS finds the optimal sum-of-costs solution on benchmark instances where the ground truth is known.
- Scalability analysis: plots showing how CBS and Prioritized A* diverge in solution quality and runtime as the number of agents grows from 2 to 20.
- A warehouse-relevant takeaway: at what fleet size does CBS become computationally prohibitive, and what does that mean for real deployment?

Success is defined as: (1) CBS matches or improves on Prioritized A* solution cost for all tested instances, and (2) runtime and CT-node count are reported as functions of agent count, enabling a clear discussion of computational trade-offs.